

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

Asignatura:	APLICACIONES WEB ORIENTADAS A SERVICIOS				
Grupo:			Fecha:	29-3/jul/2026	
Tipo de Evaluación (Marcar con una "X" según corresponda)	Diagnóstica		Unidad:	II	Calificación
	Formativa				
	Sumativa	X	Tiempo (minutos):	100	
	Recuperación				
Nombre del profesor:		Mitsiu Alejandro Carreño Sarabia			

Nombre del alumno: _____ Matrícula: _____

Esta evidencia equivale al 55% de la calificación final en la unidad 2.

Cumple los siguientes requisitos usando el framework NestJs (1 punto cada requisito)

Del ejercicio **1 al 4** usa los archivos **src/app.controller.ts** y **src/app.service.ts**

Del ejercicio **5 al 10** usa los archivos **src/tienda/tienda.controller.ts** y **src/tienda/tienda.service.ts**

1. Crear una ruta GET `"/negativo"` que responda cuántas veces se ha solicitado ese endpoint con un número negativo, es decir invocar `"negativo"` tres veces debe regresar -3 (usa un atributo en la clase service para guardar el estado del contador actual).

Request	Response	Request	Response
GET /negativo	-1	GET /negativo GET /negativo GET /negativo	-3
GET /negativo GET /negativo	-2		

2. Crea la ruta PUT `"/esPar/:num"` que convierta el valor de `"num"` a número y responda el string `"par"` o `"impar"` si num es par o no (usa el operador `%` para revisar el residuo `(num%2==0)`)

Request	Response	Request	Response
PUT /esPar/10	par	PUT /esPar/5	impar
PUT /esPar/3	impar	PUT /esPar/9	impar

3. Crea la ruta PATCH `"/max"` que lea tres query param llamados `"num1"`, `"num2"`, `"num3"` los convierta a números y responda el número más grande de los tres.

Request	Response
PATCH /max?num1=1&num2=2&num3=3	3
PATCH /max?num1=10&num2=2&num3=3	20

4. Crea la ruta POST `"/favClass"` que regrese un arreglo fijo con los elementos `["APLICACIONES", "WEB", "ORIENTADAS", "A", "SERVICIOS"]`

Request	Response
POST /favClass	<code>["APLICACIONES", "WEB", "ORIENTADAS", "A", "SERVICIOS"]</code>

5. Crea los archivos **`src/tienda/tienda.controller.ts`** y **`src/tienda/tienda.service.ts`** para manejar todas las rutas del recurso `/tienda`.
6. Haz que el archivo **`src/tienda/tienda.service.ts`** almacene un atributo `"misArticulos"` de tipo `string[]` con el valor inicial `["frijoles", "papitas", "pan", "galletas"]`.
7. Crea la ruta GET `"/tienda"` que regrese el estado actual del arreglo `"misArticulos"`
8. Crea la ruta POST `"/tienda/comprar/:index"` que convierta a número el valor de `"index"` y actualice a `"vendido"` el elemento en la posición `"index"` del arreglo `"misArticulos"` (usa la sintaxis `misArticulos[index] = "vendido"`; para actualizar el arreglo en una posición específica)

Request	Response
POST /tienda/comprar/0	<code>["vendido", "papitas", "pan", "galletas"]</code>
POST /tienda/comprar/3	<code>["frijoles", "papitas", "pan", "vendido"]</code>

9. Crea la ruta POST `"/tienda"` que lea un query param llamado `"nuevoArt"` y que agregue el valor de `nuevoArt` al arreglo `"misArticulos"` regresa la nueva lista de `"misArticulos"` (usa el método `.push()` disponible en cualquier arreglo)

Request	Response
POST /tienda?nuevoArt=cloro	<code>["frijoles", "papitas", "pan", "galletas", "cloro"]</code>
POST /tienda?nuevoArt=jabon	<code>["frijoles", "papitas", "pan", "galletas", "jabon"]</code>

10. Crea la ruta GET `"/tienda/find/:value"` que responda un booleano si `"value"` está en el arreglo `"misArticulos"` (usa el método `.includes(value)` disponible en cualquier arreglo)

Request	Response
GET /tienda/find/frijoles	true
GET /tienda/find/galletas	true
GET /tienda/find/refresco	false